

Unit 5 Analytical Answers

192521079

Q1. Major Security Threats in Web Communication and Mitigation

Threats to the four core security services:

Security goal	Typical attack	Mitigation
Confidentiality	Eavesdropping/sniffing on network traffic, man-in-the-middle (MITM)	Encryption (symmetric ciphers like AES within TLS sessions)
Integrity	Packet tampering, data modification in transit	Hash-based Message Authentication Codes (HMAC), digital signatures
Authentication	Impersonation, phishing, spoofed servers	Digital certificates issued by trusted CAs, public-key authentication
Non-repudiation	A party denying having sent/received a message or transaction	Digital signatures bound to a private key only the sender possesses

Analysis: Web communication over plain HTTP is vulnerable at every layer — an attacker on a shared network (public Wi-Fi, compromised router) can read, modify, or inject data. Security protocols like SSL/TLS combine these defenses: asymmetric cryptography during the handshake authenticates the server (and optionally the client) and safely exchanges a session key; symmetric encryption then protects confidentiality for the bulk of the session (efficient for large data volumes); integrity is enforced via MACs computed over every record, so any tampering is detected; and digital signatures (e.g., on certificates, or in protocols like SET) provide non-repudiation, since only the holder of a private key could have produced a valid signature. No single mechanism covers all four goals — this is why modern web security is a layered combination of encryption, hashing, PKI, and signatures rather than one algorithm.

Q2. SSL vs TLS

Aspect	SSL	TLS
Architecture	Same layered design (Record + Handshake + Alert + Change Cipher Spec) — TLS evolved directly from SSL 3.0	Same architecture, refined
Handshake	Uses weaker MAC construction (MAC based on MD5/SHA-1 combined ad hoc)	Uses HMAC (formally proven construction), supports stronger/extensible cipher suites
Cryptographic algorithms	Supports outdated/weak ciphers (RC4, DES, export-grade ciphers), MD5	Supports modern AES, ChaCha20, SHA-2/SHA-3-based suites; TLS 1.3 removes legacy weak ciphers entirely
Key derivation	Weaker PRF	Stronger, standardized PRF (and in TLS 1.3, a modern HKDF-based scheme)
Resistance to attacks	Vulnerable to POODLE, BEAST, and other protocol-level attacks	Progressively hardened; TLS 1.2 fixed most SSL issues, TLS 1.3 removes renegotiation and legacy handshake modes that enabled many attacks entirely
Handshake speed	Full round trips only	TLS 1.3 supports 1-RTT and 0-RTT resumption, faster and more secure

Why TLS is preferred: SSL (all versions, including SSL 3.0) is formally deprecated and considered insecure

— it is vulnerable to attacks like POODLE that let an attacker downgrade and decrypt sessions. TLS was designed as SSL's direct successor specifically to close these cryptographic and protocol-design weaknesses, uses stronger, well-analyzed cryptographic primitives, and is the protocol mandated by current security standards (PCI-DSS, browser vendor policy) for any web traffic requiring confidentiality or integrity.

Q3. The SSL/TLS Handshake Process

Goal: Let a client and server agree on protocol version, cipher suite, and a shared session key, while authenticating the server (and optionally the client), before any application data is exchanged.

Simplified flow (TLS 1.2-style, for clarity):

1. **ClientHello:** Client sends supported TLS versions, cipher suites, and a random value (client random).
2. **ServerHello:** Server picks a protocol version and cipher suite, sends its own random value (server random).
3. **Certificate:** Server sends its digital certificate (containing its public key, signed by a trusted Certificate Authority).
4. **Server Key Exchange / Certificate verification:** Client verifies the certificate chain up to a trusted root CA, checks validity dates and hostname match.
5. **Key exchange:** Using algorithms like RSA or (preferably) Diffie-Hellman/ECDHE, the client and server derive a shared **pre-master secret**, from which both independently compute the same **session key** (symmetric key) using the exchanged random values.
6. **Finished messages:** Both sides send a MAC-protected "Finished" message encrypted with the new session key, confirming the handshake succeeded and wasn't tampered with.
7. **Secure session:** All further application data is encrypted and integrity-protected using the negotiated symmetric session key.

Role of certificates: They bind a public key to an identity (e.g., a domain name), verified by a CA's digital signature. This lets the client trust that it is really talking to the intended server, not an impersonator.

Role of session keys: Asymmetric cryptography (used to establish trust and exchange secrets) is computationally expensive, so it's used only briefly during the handshake. The resulting symmetric session key is fast and used to encrypt/authenticate the actual data flow for the rest of the session — this hybrid approach balances strong authentication with efficient bulk encryption.

Q4. SSL/TLS Protecting Online Banking Credentials

Scenario: A user logs into a banking site over an unsecured (e.g., public Wi-Fi) network.

Analysis:

- **Against eavesdropping:** Once the TLS handshake completes, all data — login credentials, account numbers, transaction details — is encrypted with a symmetric session key. An attacker passively capturing packets on the same network sees only ciphertext, which is computationally infeasible to decrypt without the session key.
 - **Against man-in-the-middle attacks:** The server's identity is verified through its CA-signed certificate during the handshake. If an attacker tries to intercept and impersonate the bank's server, they cannot produce a certificate that will validate against a trusted root CA for the bank's real domain, so the browser will reject the connection with a security warning (unless the user ignores it — see Q5).
 - **Integrity protection:** Every record is protected with a MAC, so any attempt to alter transaction data in transit (e.g., changing the payee's account number) invalidates the MAC and the tampered packet is rejected.
 - **Conclusion:** Even on a completely untrusted network, TLS ensures that credentials and transaction data remain confidential, unmodified, and exchanged only with a properly authenticated server — the security guarantee depends on the network layer being untrusted by design, which is exactly the threat model TLS was built for.
-

Q5. Digital Certificates in SSL/TLS and Risks of Accepting Invalid Ones

Role of certificates: A digital certificate binds a public key to an entity's identity, digitally signed by a Certificate Authority (CA) that the client's browser/OS already trusts. During the handshake, the client validates: (a) the certificate's signature chains up to a trusted root CA, (b) the certificate hasn't expired, (c) it hasn't been revoked (via CRL/OCSP), and (d) the domain name matches.

Risks of accepting an invalid/expired/self-signed certificate without verification:

- **Impersonation / MITM:** An attacker can present a self-signed or fraudulently obtained certificate for a spoofed server. If the client accepts it blindly, it will establish an encrypted session — but encrypted *with the attacker*, not the real server, defeating the entire purpose of TLS.
- **Expired certificates:** May indicate the server operator has stopped maintaining the certificate/key properly, increasing risk that the private key has been mishandled or the domain's ownership has changed since issuance.
- **Loss of non-repudiation and trust:** Without proper CA validation, there is no assurance that "the server I'm talking to" is who it claims to be, undermining authentication entirely and enabling credential theft, data manipulation, and phishing-style attacks disguised as encrypted (and therefore falsely "trusted-looking") connections.
- **Conclusion:** Certificate validation is not optional — it is the mechanism that ties cryptographic trust to real-world identity. Skipping it converts TLS from a genuine security guarantee into a false sense of security.

Q6. Symmetric vs Asymmetric Cryptography in SSL/TLS

Aspect	Symmetric	Asymmetric
Key structure	Single shared secret key	Public/private key pair
Speed	Fast, efficient for bulk data	Computationally expensive
Used for	Encrypting the actual application data (record layer)	Authentication and secure key exchange during the handshake
Examples in TLS	AES, ChaCha20	RSA, Diffie-Hellman/ECDHE, digital signatures

Why both are required: Asymmetric cryptography solves the *key distribution problem* — two parties who have never met can authenticate each other and agree on a shared secret over an insecure channel without ever needing to have exchanged that secret beforehand. However, it's far too slow to encrypt large volumes of ongoing traffic. Symmetric cryptography is extremely fast but requires both parties to already share a secret key, which is the very problem asymmetric crypto just solved. TLS therefore uses asymmetric cryptography briefly (handshake: authenticate the server, negotiate a shared session key) and then switches to symmetric cryptography for the bulk of the session (fast, ongoing encryption/decryption of application data) — a "hybrid cryptosystem" that gets the security benefits of asymmetric crypto and the performance of symmetric crypto.

Q7. Security Objectives Achieved by TLS

- **Confidentiality:** Achieved via symmetric encryption (e.g., AES) of the record layer using the session key derived during the handshake — an eavesdropper sees only ciphertext.
- **Integrity:** Each TLS record includes a MAC (HMAC in TLS 1.2, an AEAD-integrated tag in TLS 1.3's AES-GCM/ChaCha20-Poly1305), so any modification of data in transit is detectable and the record is rejected.
- **Authentication:** The server (and, for mutual-TLS, the client) presents a CA-signed certificate proving its identity; the handshake's Finished messages also confirm neither party's messages were tampered with mid-negotiation.
- **Secure key exchange:** Algorithms like Diffie-Hellman (especially ephemeral variants, ECDHE) allow both parties to derive a shared secret over a public channel such that even someone recording the

entire handshake cannot compute the session key without solving a computationally hard problem (discrete logarithm). Ephemeral key exchange also provides **forward secrecy** — compromise of a long-term private key later doesn't expose past session keys.

Together these four objectives are what let TLS provide a genuinely secure channel over an inherently insecure network like the public internet.

Q8. Attacker Intercepting a TLS Handshake

Possible attacks if an attacker sits between browser and server during the handshake:

- **Man-in-the-middle impersonation:** Attempting to present a forged/self-signed certificate as if it belonged to the real server.
- **Downgrade attacks:** Trying to force the negotiation to a weaker, older protocol version or cipher suite that is easier to break (e.g., forcing SSL 3.0 to exploit POODLE).
- **Replay attacks:** Capturing handshake messages to attempt reuse in a later session.
- **Passive eavesdropping on the handshake:** Attempting to derive the session key from captured handshake data alone.

How TLS mechanisms prevent these:

- **Certificate validation** stops impersonation: the browser will refuse to trust a certificate not properly signed by a recognized CA for the correct domain, so the attacker's fake certificate is rejected and the connection fails (or the browser shows a warning).
 - **Cipher suite negotiation integrity:** The Finished messages at the end of the handshake include a MAC over the *entire* handshake transcript, computed with the newly derived session key. If an attacker tampered with earlier handshake messages (e.g., to force a downgrade), the transcript hashes won't match and the handshake aborts — this specifically defeats downgrade attacks.
 - **Ephemeral key exchange (ECDHE):** Even if an attacker records the whole handshake, they cannot derive the shared secret without solving the discrete-log problem, since the ephemeral private keys are never transmitted and are discarded after use — this also defeats passive eavesdropping and limits the value of any later key compromise (forward secrecy).
 - **Nonces/random values:** Client and server randoms are mixed into key derivation, preventing simple replay of a captured handshake.
-

Q9. Architecture and Transaction Flow of SET (Secure Electronic Transaction)

Participants: Cardholder (customer), Merchant, Issuer (cardholder's bank), Acquirer (merchant's bank), Payment Gateway (bridges the internet transaction to the traditional card payment network), and a Certificate Authority hierarchy that issues certificates to all parties.

Transaction flow (simplified):

1. Customer selects goods and initiates checkout; the merchant's software and the customer's SET-enabled wallet establish a session.
2. The customer sends a **Purchase Order (PO)** to the merchant containing order information, and separately (encrypted so the merchant cannot read it) **Payment Information (PI)** intended only for the payment gateway/bank.
3. The customer generates a **dual signature** (see Q12) linking the PO and PI without revealing card details to the merchant or order details to the bank.
4. The merchant forwards the encrypted payment information to the **payment gateway**, which decrypts it, verifies the dual signature, and forwards an authorization request to the **issuer bank** through existing card payment networks.
5. The issuer approves/declines; the gateway relays authorization back to the merchant.
6. The merchant completes the order and ships goods, later requesting **capture/settlement** of funds through the acquirer.

How SET secures the transaction: SET uses digital certificates to authenticate every party (customer, merchant, gateway), and encryption + dual signatures to ensure the merchant never sees the customer's raw card number and the bank never sees the order details — each party sees only the information relevant to its role, while cryptographic bindings still guarantee consistency between the two halves of the transaction.

Q10. SSL/TLS vs SET

Aspect	SSL/TLS	SET
Authentication	Typically authenticates the server only (client auth optional/rare)	Authenticates customer, merchant, <i>and</i> payment gateway via certificates issued to all parties
Confidentiality	Encrypts the entire communication channel	Selectively hides payment details from the merchant and order details from the bank (via dual signatures), rather than blanket channel encryption
Integrity	MAC/AEAD on the transport channel	Digital signatures on transaction-specific data (dual signature construction)
Payment security	General-purpose; doesn't understand payment semantics — application must handle card data securely on top of TLS	Purpose-built for card payments; enforces separation of concerns between merchant and bank
Practical implementation	Extremely widely deployed — built into every browser/server; low overhead, fast handshake	Complex PKI requirements (certificates for every customer and merchant), heavier processing; largely failed to gain adoption and has been effectively superseded

Which is more suitable for general web communication? SSL/TLS, because it is a general-purpose, transport-level security protocol that is lightweight, universally supported, and doesn't require every user to obtain a personal certificate — a practical requirement that made SET too cumbersome for mass adoption. SET's fine-grained, payment-specific security model was conceptually strong but too heavyweight for the web at large; today, TLS combined with tokenization standards (e.g., PCI-DSS-compliant payment processors, 3-D Secure) fills the role SET was originally designed for.

Q11. SET Protecting Customer, Merchant, and Payment Gateway

Scenario: A customer sends payment info to a merchant in an online purchase.

Analysis:

- **Digital certificates** issued by a trusted CA authenticate all three parties: the customer's certificate proves they hold a legitimate payment card account (tied to the issuing bank), the merchant's certificate proves their registered business identity, and the payment gateway's certificate authenticates the bank-side processing endpoint. This mutual authentication prevents impersonation at any stage.
- **Digital signatures** ensure integrity and non-repudiation: the customer signs the order/payment data so neither party can later deny the transaction occurred as recorded, and the merchant cannot alter the payment amount without invalidating the signature.
- **Encryption + dual signature separation** ensures the merchant only receives the *order information* (not the card number), while the payment gateway only receives the *payment information* (not the order details) — protecting the customer's card data from merchant exposure/breach risk, while still cryptographically linking the two halves so the bank can be sure it's authorizing payment for the exact order the merchant is fulfilling.
- **Net effect:** Each party is authenticated, sees only the data it needs, and cannot repudiate its role in

the transaction — a much stronger and more compartmentalized trust model than a single end-to-end encrypted TLS channel provides on its own.

Q12. Dual Signatures in SET

Concept: A dual signature cryptographically links two separate pieces of information — the **Order Information (OI)**, meant for the merchant, and the **Payment Information (PI)**, meant for the payment gateway/bank — without either party seeing the other's data, while still allowing verification that both belong to the same transaction.

How it works:

1. The customer computes a hash of the OI and a hash of the PI separately.
2. These two hashes are concatenated and hashed again to form a combined digest.
3. The customer signs this combined digest with their private key — this signed value is the **dual signature**.
4. The merchant receives the OI, the PI's hash, and the dual signature: they can verify the dual signature using the customer's public key and the *hash* of the PI (without ever seeing the actual payment data), confirming the OI is legitimately linked to that particular payment.
5. The bank/payment gateway receives the PI, the OI's hash, and the dual signature: they can verify the same way, confirming the PI belongs to that specific order — without ever seeing the order contents.

Why this matters:

- **Confidentiality:** Each party only ever handles the data relevant to its role — the merchant never sees card details, and the bank never sees the purchase details (which items were bought, etc.), limiting exposure if either party is compromised.
- **Integrity:** Because both pieces of data are bound together through a shared cryptographic digest and a single signature, neither the OI nor the PI can be altered or mismatched (e.g., substituting a different payment for a different order) without invalidating the signature.
- **Non-repudiation:** The signature, tied to the customer's private key, proves the customer authorized exactly this order/payment pairing.

Q13. Viruses vs Worms vs Trojans

Criterion	Virus	Worm	Trojan
Propagation mechanism	Attaches to and spreads via infected files/programs; requires the infected file to be executed or shared	Self-propagates autonomously across networks, exploiting vulnerabilities or misconfigurations	Does not self-replicate; relies on social engineering to trick a user into installing it (disguised as legitimate software)
Dependency on host program	Requires a host file/program to attach to and cannot run independently	Independent, standalone program — no host file needed	Independent standalone program, but disguised as something benign/useful
Detection challenges	Signature-based AV can catch known variants; polymorphic viruses evade static signatures	Network-level anomaly detection needed (unusual traffic spikes); fast spread makes containment time-critical	Often evades detection because it appears to be a legitimate, user-installed application; may not trigger typical malware heuristics until it acts
Potential damage	Corrupts/destroys files, can spread when infected files are shared	Can rapidly exhaust network bandwidth and system resources across an entire network, causing widespread outages	Opens backdoors, steals data, grants remote unauthorized access, often used to establish a foothold for further attacks

Q14. University Network — Traffic Spike and Slowdowns

Analysis: The described symptoms — rapid, wide traffic increase and multiple systems slowing down simultaneously across the network — most closely match a **worm**.

Justification:

- Worms are self-replicating and self-propagating: once one host is infected, the worm actively scans the network and infects other vulnerable machines *without* needing any user action, which explains the *rapid, network-wide* spread rather than a slow, file-sharing-dependent infection pattern typical of a virus.
- The heavy traffic itself is a classic worm signature — each infected host tries to find and infect more hosts, generating large volumes of scanning/propagation traffic that saturates bandwidth and degrades performance across many machines simultaneously, matching “multiple system slowdowns.”
- A Trojan would typically cause suspicious activity on individual infected machines (data exfiltration, remote access) rather than a network-wide, simultaneous traffic surge.
- A virus’s spread is generally slower and tied to file execution/sharing, and wouldn’t typically cause an immediate, sudden network-wide traffic spike on its own.

Conclusion: The incident’s *rapid, network-wide, simultaneous* nature is the strongest indicator of a **worm** outbreak, and the university’s IT team should isolate affected network segments immediately to contain further spread.

Q15. How Worms Spread and Prevention Measures

How worms spread without significant user interaction: Worms exploit software vulnerabilities (e.g., unpatched services, weak network protocols) or misconfigurations to gain remote code execution on a target machine automatically. Once resident, the worm scans the local network (and beyond) for other machines exposing the same vulnerability and replicates itself onto them — this cycle can repeat exponentially, allowing a worm to spread to thousands of machines within minutes to hours, entirely without any user clicking a link or opening a file.

Suggested security measures:

- **Patch management:** Promptly apply security patches to close the vulnerabilities worms exploit.
 - **Network segmentation:** Divide the network into isolated zones/VLANs so a worm’s spread is contained rather than reaching the entire organization.
 - **Firewalls and IDS/IPS:** Block or flag anomalous scanning/propagation traffic patterns characteristic of worm behavior.
 - **Disable unnecessary services/ports:** Reduces the attack surface available for exploitation.
 - **Network traffic monitoring:** Detect sudden traffic spikes early to enable rapid isolation of infected segments.
 - **Endpoint protection with behavioral detection:** Catches worm activity even when signature-based detection fails against new/unknown variants.
-

Q16. Trojan via a “Legitimate” Application

Scenario: A user downloads what appears to be a legitimate app, which secretly grants unauthorized access.

Analysis: This is a classic **Trojan horse** attack. The malicious payload is disguised inside software that appears useful or trustworthy, relying entirely on the user’s own action (downloading and running it) rather than any self-propagation mechanism — the user is socially engineered into installing the malware themselves.

Distinguishing characteristics of a Trojan vs virus/worm:

- **No self-replication:** Unlike viruses and worms, a Trojan does not copy itself into other files or spread across the network automatically — its only “propagation” is the initial deceptive installation.

- **No host-file dependency:** Unlike a virus, it doesn't need to attach itself to another program; it runs as its own standalone (if disguised) application.
- **Deception-based delivery:** Its defining trait is masquerading as something desirable/benign (a game, utility, "cracked" software, etc.) to get the user to install it voluntarily — this is fundamentally different from a worm's automatic network-exploitation-based spread.
- **Payload behavior:** Often creates backdoors, logs keystrokes, or exfiltrates data — providing ongoing unauthorized access rather than simply damaging/corrupting files (though it can do that too).

Q17. Role of Firewalls in Protecting Internal Networks

Analysis: A firewall acts as a controlled checkpoint between an organization's trusted internal network and untrusted external networks (like the internet), enforcing a security policy on what traffic is allowed to pass.

- **Packet filtering:** Examines each packet's header (source/destination IP, port, protocol) against a rule set, blocking traffic that doesn't match allowed patterns — a first line of defense against unauthorized external access attempts.
- **Access control:** Firewalls enforce policies about which internal services are reachable from outside (and vice versa), e.g., allowing inbound traffic only to a public web server on port 443 while blocking direct access to internal database servers.
- **Traffic monitoring:** Logging and analyzing traffic passing through the firewall helps detect suspicious patterns (e.g., repeated connection attempts, port scans) and provides an audit trail for incident investigation.

Overall contribution: Together, these functions reduce the attack surface exposed to external threats, enforce the principle of least privilege for network access, and give administrators visibility into what's crossing the network boundary — forming a foundational (though not sufficient on its own) layer of network defense.

Q18. Comparing Firewall Types

Type	How it works	Advantages	Limitations
Packet-filtering	Examines individual packet headers (IP, port, protocol) against static rules, no awareness of connection state	Fast, low overhead, simple to implement	No context of ongoing connections; vulnerable to spoofing and cannot detect application-layer attacks
Stateful inspection	Tracks the state of active connections (e.g., TCP handshake state), allows only packets that fit an established, legitimate session	More secure than stateless filtering; can detect out-of-state or spoofed packets	Still limited to network/transport-layer visibility; doesn't inspect application payload content
Proxy / application-level	Acts as an intermediary, fully terminating and re-establishing connections; inspects traffic at the application layer (e.g., HTTP content)	Deep inspection of application data, can filter malicious content/commands, hides internal network structure	Higher latency/overhead, can be a performance bottleneck, needs a proxy per application protocol
Next-generation firewall (NGFW)	Combines stateful inspection with deep packet inspection, application awareness, intrusion prevention, and often user identity-based rules	Most comprehensive protection; can detect application-layer threats, malware signatures, and enforce granular policies	More complex to configure and maintain, higher cost, and greater processing overhead than simpler firewall types

Q19. Firewall Architecture for a Multi-Server Organization

Scenario: Protecting a web server, mail server, database server, and internal employee network.

Design:

- **DMZ (Demilitarized Zone):** Place the publicly-facing **web server** and **mail server** (specifically its externally-facing components, e.g., SMTP gateway) in a DMZ — a network segment isolated by firewalls on both sides, separate from the internal network. This allows external users to reach these services without giving them any direct path into the internal network.
- **External firewall:** Sits between the internet and the DMZ, allowing only necessary inbound traffic (e.g., HTTPS to the web server, SMTP to the mail server) and blocking everything else.
- **Internal firewall:** Sits between the DMZ and the internal network, allowing only tightly controlled traffic from DMZ servers to internal resources they legitimately need (e.g., the web server may need limited access to the database server for application data, but nothing else).
- **Database server placement:** Kept in the **internal network** (or an even more restricted internal segment), never directly exposed to the internet — the web server in the DMZ acts as an intermediary, so a compromise of the web server doesn't automatically grant direct database access.
- **Internal employee network:** Fully separated from the DMZ, protected by the internal firewall, with its own outbound-only or tightly controlled internet access policy.

Justification: This layered “defense-in-depth” design ensures that even if a public-facing server in the DMZ is compromised, the attacker still faces another firewall boundary before reaching sensitive internal systems like the database or employee workstations — no single point of compromise gives full network access.

Q20. Stateful vs Stateless Firewalls for TCP Connections

Stateless (packet-filtering) firewall: Evaluates each packet independently against static rules, with no memory of previous packets. For TCP, it can only filter based on header fields like source/destination port — it cannot tell whether a packet legitimately belongs to an already-established connection or is a rogue/spoofed packet claiming to be part of one.

Stateful firewall: Maintains a connection state table tracking active TCP sessions (e.g., tracking the three-way handshake: SYN, SYN-ACK, ACK) and only permits packets that are consistent with a legitimate, already-tracked connection state.

Why stateful inspection is stronger:

- It can detect and drop packets that don't correspond to any real, established connection (e.g., an attacker sending an ACK packet with no prior SYN, common in certain scanning/spoofing techniques).
 - It naturally allows return traffic for outbound connections initiated internally, without needing broad static rules that could otherwise be exploited (stateless firewalls often must keep wide-open high-numbered ports for return traffic, an attack surface stateful tracking avoids).
 - It provides better protection against attacks that rely on crafting individual packets to slip past header-only rules, since context (the full connection state) is required to be accepted.
-

Q21. Why a Firewall Alone May Not Be Sufficient

Scenario: A firewall blocks unauthorized traffic, but an attacker compromises a system using *permitted* traffic.

Analysis: Firewalls make access-control decisions based on traffic characteristics (ports, protocols, sometimes application awareness) — but they generally trust traffic that matches an allowed rule (e.g., HTTP/HTTPS to a web server) without deeply analyzing whether that traffic's *content* is malicious. An attacker can exploit this by hiding an attack inside otherwise legitimate-looking traffic — e.g., a SQL injection or malicious payload sent over an allowed HTTPS connection to a web application. Since the firewall's job is (mostly) to filter based on *permission*, not *payload intent*, it has no way to recognize the attack.

How IDS/IPS complements a firewall:

- An **IDS (Intrusion Detection System)** monitors traffic (and/or host activity) for known attack signatures or anomalous behavior patterns, even within traffic the firewall has already allowed through, and alerts administrators.
- An **IPS (Intrusion Prevention System)** goes further, actively blocking or terminating connections identified as malicious in real time, providing active response rather than just detection.
- Together, firewall + IDS/IPS forms a layered defense: the firewall restricts *what* traffic is allowed in at all, while IDS/IPS inspects the *behavior/content* of that allowed traffic for signs of compromise — addressing exactly the gap where permitted-but-malicious traffic slips past pure access-control mechanisms.

Q22. IDS vs IPS

Aspect	IDS	IPS
Deployment	Typically deployed out-of-band (passive, monitors a copy of traffic, e.g., via a network tap/SPAN port)	Deployed in-line (traffic passes directly through it)
Detection	Detects suspicious activity using signatures/anomaly analysis	Same detection techniques, but tightly coupled to a real-time blocking mechanism
Response	Passive — generates alerts for administrators to investigate and respond manually	Active — can automatically drop packets, reset connections, or block offending IPs in real time
Traffic handling	Does not sit in the traffic path; no impact on latency, but also cannot stop an attack in progress	Sits directly in the traffic path; can stop attacks as they happen, but introduces potential latency and a single point of failure if misconfigured
Impact on network security	Improves visibility and forensic capability, supports incident response	Provides active, real-time protection, reducing the window of exposure, but a false positive can disrupt legitimate traffic

Q23. Signature-Based vs Anomaly-Based IDS

Aspect	Signature-based	Anomaly-based
Method	Matches traffic/behavior against a database of known attack patterns	Builds a baseline of “normal” behavior and flags significant deviations
Effectiveness against known attacks	Very effective — high accuracy for previously catalogued attacks	Less precise for known attacks compared to a direct signature match, but still can catch them if they deviate from baseline
Effectiveness against zero-day attacks	Poor — cannot detect attacks with no existing signature	Better — can flag novel attacks purely because their behavior is abnormal, even without prior knowledge of the specific attack
False positives	Generally low, since matches are specific	Higher — legitimate but unusual behavior can be flagged incorrectly
False negatives	Higher for new/modified attacks (evasion via slight variation defeats exact signature matching)	Lower for novel attacks, but depends heavily on how well the “normal” baseline was established

Overall trade-off: Signature-based detection is precise and reliable for known threats but blind to new ones; anomaly-based detection can catch novel/zero-day attacks but at the cost of a higher false-positive rate and dependency on an accurate baseline of normal behavior. Many modern systems combine both

approaches for layered coverage.

Q24. Detecting Failed Logins + Unusual Traffic

Scenario: Repeated unsuccessful login attempts followed by unusual network traffic from an internal computer.

How an IDS could detect this:

- **Signature-based detection** could flag the repeated failed login attempts as matching a known brute-force/credential-stuffing attack pattern (e.g., exceeding a threshold of failed attempts within a short time window).
- **Anomaly-based detection** would notice that the subsequent network traffic pattern from that host deviates significantly from its normal baseline (e.g., a workstation suddenly making unusual outbound connections, scanning other internal hosts, or transferring large volumes of data) — consistent with a successful compromise followed by lateral movement or data exfiltration.
- The IDS correlates these two events — failed logins followed by abnormal traffic — as a strong indicator that an attacker eventually succeeded (e.g., via a successful guess, credential reuse, or malware) and is now conducting further malicious activity.

How an IPS could respond:

- Automatically **block the source IP** attempting the repeated logins after exceeding a failure threshold, preventing further brute-force attempts.
 - **Terminate or quarantine** the suspicious internal host's network connections if the subsequent traffic matches known attack/exfiltration signatures, containing the potential compromise before it spreads further or exfiltrates more data.
 - **Alert security teams in real time** while enforcing containment, allowing for rapid manual investigation and remediation (e.g., isolating and re-imaging the affected machine).
-

Q25. Comprehensive Web Security Architecture for Online Banking

Design combining TLS, digital certificates, firewalls, IDS/IPS, and malware protection:

1. **TLS (with strong cipher suites, TLS 1.2/1.3 only):** Encrypts all traffic between customers' browsers and the bank's web servers, protecting login credentials and transaction data from eavesdropping and tampering (see Q3-Q4).
2. **Digital certificates (EV/OV certificates from trusted CAs):** Authenticate the bank's identity to customers, preventing impersonation/phishing sites from convincingly posing as the real bank; certificate pinning can add an extra layer against fraudulent certificates.
3. **Firewall architecture with DMZ:** Public-facing web/application servers sit in a DMZ, isolated from core banking databases and internal systems by both external and internal firewalls, following the layered design discussed in Q19 — ensuring a compromised web-facing component cannot directly reach sensitive account data.
4. **IDS/IPS:** Monitors all traffic crossing the firewall boundaries and within the internal network for known attack signatures (SQL injection attempts, brute-force logins) and anomalous behavior (unusual transaction patterns, insider threats), actively blocking malicious traffic in real time (see Q21-Q24).
5. **Malware protection (endpoint + network-level):** Anti-malware/EDR (Endpoint Detection and Response) on all servers and employee workstations detects and contains viruses, worms, and Trojans before they can compromise systems handling sensitive financial data (see Q13-Q16); regular patching closes vulnerabilities worms could exploit.
6. **Additional layers:** Multi-factor authentication for customer logins, strict access control and least-privilege policies for internal staff, regular security audits/penetration testing, and encrypted, access-controlled database storage for account data at rest.

Analysis of how each mechanism contributes: No single control is sufficient on its own — TLS and certificates secure the communication channel and authenticate identity but say nothing about server/network compromise; firewalls restrict what can reach sensitive systems but can't inspect payload

intent; IDS/IPS catches attacks that slip through allowed traffic; and malware protection guards against threats introduced through other vectors (infected attachments, compromised employee devices) entirely outside the network perimeter. Together, these layers implement **defense in depth** — a foundational principle for any system, like online banking, where the cost of a single point of failure is unacceptably high.